Giải thích chi tiết: `memlayout.h`, `vm.c`, `kalloc.c`

Tài liệu này giải thích ba file quản lý bộ nhớ của xv6-riscv:

File	Vai trò
kernel/memlayout.h	Bản đồ bộ nhớ: địa chỉ vật lý của thiết bị, RAM, và địa chỉ ảo đặc biệt (trampoline, trapframe, kernel stack)
kernel/kalloc.c	Bộ cấp phát **trang vật lý** (mỗi lần 4096 byte)
kernel/vm.c	Mã **bộ nhớ ảo**: tạo/duyệt/sửa page table, map/unmap, copy dữ liệu giữa kernel và user

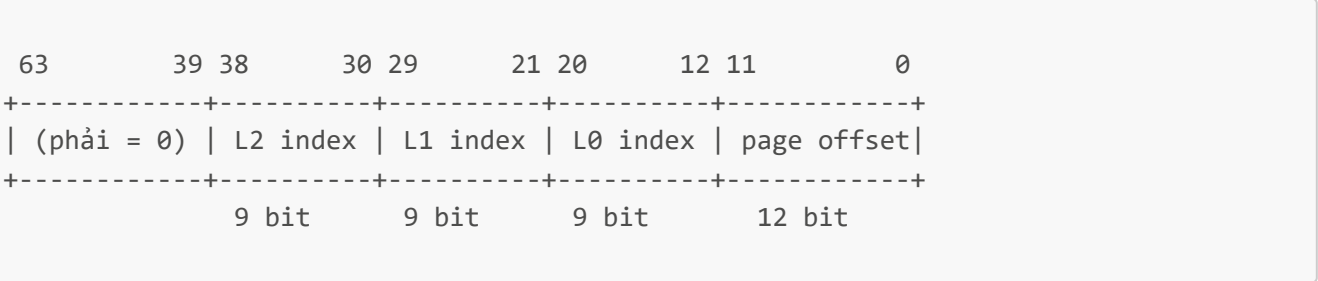
Thứ tự đọc hợp lý: `memlayout.h` (có gì ở đâu) → `kalloc.c` (lấy trang vật lý từ đâu) → `vm.c` (gắn trang vật lý vào địa chỉ ảo).

Các macro dùng nhiều (`PGSIZE`, `PTE_*`, `PX`, `PA2PTE`, ...) nằm trong kernel/riscv.h, được tóm tắt ở phần 0.

---

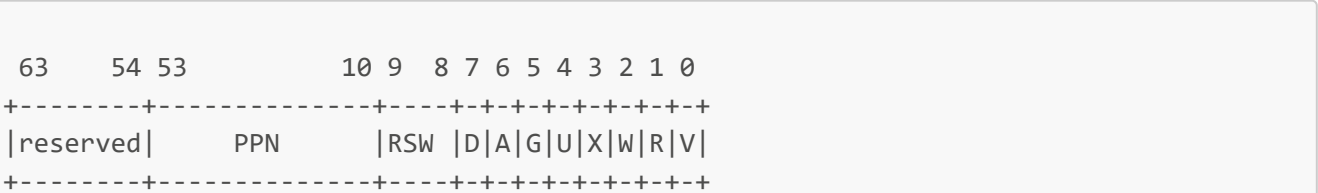
0. Kiến thức nền: Sv39 và các macro trong `riscv.h`

RISC-V dùng sơ đồ phân trang **Sv39**: địa chỉ ảo 39 bit, page table 3 cấp, trang 4 KiB.



- Mỗi page table là **một trang 4096 byte** chứa **512 PTE** (mỗi PTE 8 byte → 512 × 8 = 4096).
- Dịch địa chỉ: CPU lấy `satp` → page table gốc (cấp 2) → dùng `L2 index` chọn PTE → PTE trỏ sang page table cấp 1 → `L1 index` → page table cấp 0 → `L0 index` → PTE lá chứa số trang vật lý (PPN). Cộng `offset` vào là ra địa chỉ vật lý.

Định dạng một PTE (64 bit):



Macro (`riscv.h`)	Ý nghĩa
`PGSIZE` = 4096, `PGSHIFT` = 12	Kích thước trang, số bit offset

Macro (`riscv.h`)	Ý nghĩa
`PGROUNDUP(a)` / `PGROUNDDOWN(a)`	Làm tròn lên / xuống bội của `PGSIZE`
`PTE_V`	Valid: PTE có hiệu lực
`PTE_R`, `PTE_W`, `PTE_X`	Cho phép đọc / ghi / thực thi
`PTE_U`	Code ở user mode được phép truy cập trang này
`PX(level, va)`	Lấy 9 bit index của `va` ở cấp `level` (`(va >> (12+9*level)) & 0x1FF`)
`PA2PTE(pa)`	`(pa >> 12) << 10`: đặt số trang vật lý vào bit 10..53 của PTE
`PTE2PA(pte)`	`(pte >> 10) << 12`: phép ngược lại
`PTE_FLAGS(pte)`	`pte & 0x3FF`: 10 bit cờ thấp
`MAXVA`	`1 << 38`: địa chỉ ảo lớn nhất xv6 dùng (kém 1 bit so với 39 bit Sv39 cho phép, để khỏi phải xử lý sign-extension)
`MAKE_SATP(pt)`	`SATP_SV39 | (pt >> 12)`: giá trị ghi vào thanh ghi `satp` để bật Sv39 và trỏ tới page table `pt`

Quy ước quan trọng: một PTE hợp lệ mà **không có** bit R/W/X nào là PTE **trỏ tới page table cấp thấp hơn**; có ít nhất một bit R/W/X là **PTE lá** (trỏ tới trang dữ liệu). `freewalk()` và `uvmunmap()` dựa vào quy ước này.

---

1. `kernel/memlayout.h` — bản đồ bộ nhớ

File chỉ chứa `#define`, không có code thực thi. Nó cố định "cái gì nằm ở địa chỉ nào" cho cả kernel lẫn user.

1.1 Địa chỉ **vật lý** của máy QEMU `virt`

Comment đầu file mô tả bản đồ vật lý do QEMU (`hw/riscv/virt.c`) dựng:

Địa chỉ vật lý	Thành phần	Macro
`0x00001000`	Boot ROM của QEMU	—
`0x02000000`	CLINT (timer, ngắt phần mềm)	— (xv6 truy cập qua CSR nên không cần macro)
`0x0C000000`	PLIC (bộ điều khiển ngắt ngoài)	`PLIC`
`0x10000000`	UART0 (cổng nối tiếp, console)	`UART0`, `UART0_IRQ` = 10
`0x10001000`	Đĩa virtio	`VIRTIO0`, `VIRTIO0_IRQ` = 1
`0x80000000`	Bắt đầu RAM; boot ROM nhảy vào đây ở machine mode, `-kernel` nạp kernel tại đây	`KERNBASE`
`0x88000000`	Cuối RAM mà kernel dùng	`PHYSTOP`

Các thiết bị (UART, virtio, PLIC) là **memory-mapped I/O**: đọc/ghi vào địa chỉ đó là nói chuyện với thiết bị, không phải với RAM. Vì vậy kernel phải map chúng vào page table của mình (xem `kvmmake` ở phần 3).

PLIC register macros:

```
#define PLIC_PRIORITY      (PLIC + 0x0)
#define PLIC_PENDING       (PLIC + 0x1000)
#define PLIC_SENABLE(hart) (PLIC + 0x2080   + (hart)*0x100)
#define PLIC_SPRIORITY(hart) (PLIC + 0x201000 + (hart)*0x2000)
#define PLIC_SCLAIM(hart)  (PLIC + 0x201004 + (hart)*0x2000)
```

Mỗi hart (lõi CPU) có thanh ghi enable / priority threshold / claim riêng, cách nhau một bước cố định. Hậu tố `S` = supervisor mode (kernel chạy ở S-mode).

1.2 RAM

```
#define KERNBASE 0x80000000L
#define PHYSTOP  (KERNBASE + 128*1024*1024)   // = 0x88000000
```

Kernel giả định có **128 MiB RAM** từ `0x80000000` đến `0x88000000`. Con số 128 MiB khớp với tham số `-m 128M` của QEMU trong Makefile. Cách RAM được chia:

```
0x80000000  KERNBASE ── entry.S, rồi kernel text (code)
            etext    ── kernel data + bss
            end      ── bắt đầu vùng cấp phát trang (kalloc quản lý từ đây)
0x88000000  PHYSTOP  ── hết RAM
```

`etext` và `end` là ký hiệu do linker script kernel/kernel.ld sinh ra, không phải macro.

1.3 Địa chỉ **ảo** đặc biệt

Giá trị cụ thể (tính từ `MAXVA = 1<<38 = 0x4000000000`, `PGSIZE = 0x1000`):

```
#define TRAMPOLINE (MAXVA - PGSIZE)                     // 0x3FFFFFF000
#define KSTACK(p)  (TRAMPOLINE - ((p)+1) * 2*PGSIZE)    // kernel stack của
process thứ p
#define TRAPFRAME  (TRAMPOLINE - PGSIZE)                // 0x3FFFFFE000
```

TRAMPOLINE — trang cao nhất của không gian ảo. Chứa code trong kernel/trampoline.S để chuyển giữa user ↔ kernel khi có trap. Trang này được map ở **cùng một địa chỉ ảo trong cả page table của user lẫn của kernel**. Đó là điều cốt yếu: lệnh `csrw satp` đổi page table, nhưng lệnh kế tiếp vẫn phải nằm ở địa chỉ ảo hợp lệ, nên code đổi `satp` phải tồn tại ở cùng địa chỉ trong cả hai không gian.

TRAPFRAME — ngay dưới trampoline, chỉ nằm trong page table của **user**. Mỗi process có một trang trapframe riêng (`p->trapframe`) để lưu 32 thanh ghi khi trap vào kernel; trampoline ghi vào đó.

KSTACK(p) — kernel stack của process có chỉ số `p` trong `proc[]`. Mỗi process chiếm `2*PGSIZE` (một trang stack + một **trang guard** không map). Bố trí trong không gian ảo của kernel, từ cao xuống thấp:

```
0x3FFFFFF000  TRAMPOLINE          (map, R+X)
0x3FFFFFE000  [guard, không map]
0x3FFFFFD000  KSTACK(0)  stack của proc[0]   (map, R+W)
0x3FFFFFC000  [guard, không map]
0x3FFFFFB000  KSTACK(1)  stack của proc[1]
   ...
```

Guard page không có PTE hợp lệ → nếu stack tràn xuống, CPU gây page fault thay vì lặng lẽ ghi đè stack của process khác. Hàm `proc_mapstacks()` (kernel/proc.c:37) cấp phát và map các stack này, được gọi từ `kvmmake`.

1.4 Bố cục bộ nhớ user (comment cuối file)

```
0x0          text (code chương trình)
             data + bss ban đầu
             stack (kích thước cố định, có guard page bên dưới)
             heap (mở rộng được bằng sbrk)
             ...
0x3FFFFFE000 TRAPFRAME
0x3FFFFFF000 TRAMPOLINE
```

Địa chỉ ảo user bắt đầu từ 0 (khác kernel, kernel chạy ở địa chỉ ảo = địa chỉ vật lý).

---

2. `kernel/kalloc.c` — bộ cấp phát trang vật lý

Bộ cấp phát rất đơn giản: **danh sách liên kết các trang trống (free list)**. Đơn vị cấp phát luôn là **một trang 4096 byte**. Dùng cho: trang của user process, kernel stack, page-table page, pipe buffer.

2.1 Cấu trúc dữ liệu

```
struct run {
  struct run *next;
};

struct {
  struct spinlock lock;
  struct run *freelist;
} kmem;
```

Mẹo chính: **không cần bộ nhớ phụ để lưu danh sách**. Một trang đang trống không ai dùng, nên ta ghi con trỏ `next` ngay vào 8 byte đầu của chính trang đó. `struct run` chỉ là cách nhìn 8 byte đầu ấy. `kmem.freelist` trỏ tới trang trống đầu tiên; mỗi trang trống trỏ tới trang trống kế.

`kmem.lock` là spinlock bảo vệ `freelist`, vì nhiều CPU có thể gọi `kalloc`/`kfree` đồng thời.

```
kmem.freelist ─▶ [trang A | next ─▶ ] [trang B | next ─▶ ] [trang C | next=0]
```

2.2 `kinit()`

```
void kinit() {
  initlock(&kmem.lock, "kmem");
  freerange(end, (void*)PHYSTOP);
}
```

Gọi một lần lúc boot từ `main()` (kernel/main.c:19) trên hart 0. Khởi tạo lock rồi đưa **toàn bộ RAM từ `end` (ngay sau kernel) đến `PHYSTOP`** vào free list. Tức là khoảng 128 MiB − kích thước kernel ≈ gần 32 nghìn trang.

2.3 `freerange(pa_start, pa_end)`

```
p = (char*)PGROUNDUP((uint64)pa_start);
for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
  kfree(p);
```

Làm tròn `pa_start` **lên** ranh giới trang (vì `end` có thể không căn trang), rồi gọi `kfree` từng trang. Điều kiện `p + PGSIZE <= pa_end` đảm bảo chỉ thêm những trang **nằm trọn** trong vùng. Đây là lý do `kfree` được dùng để "nạp" ban đầu: comment ở `kfree` ghi rõ đây là ngoại lệ — các trang này chưa từng do `kalloc` trả ra.

2.4 `kfree(pa)`

```
if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
  panic("kfree");
#ifndef LAB_SYSCALL
  memset(pa, 1, PGSIZE);       // đổ rác để bắt dangling reference
#endif
r = (struct run*)pa;
acquire(&kmem.lock);
r->next = kmem.freelist;       // chèn vào ĐẦU danh sách
kmem.freelist = r;
release(&kmem.lock);
```

1. **Kiểm tra hợp lệ** (panic nếu sai): `pa` phải căn trang, không nằm trong vùng kernel (`< end`), và nằm trong RAM (`< PHYSTOP`). Bắt được lỗi giải phóng nhầm địa chỉ.
2. **(Debug)** ghi `0x01` đè lên cả trang. Nếu có code vẫn giữ con trỏ tới trang đã free và đọc nó, sẽ thấy rác `0x0101...` và lỗi lộ ra sớm thay vì âm thầm đọc dữ liệu cũ.
3. **Chèn vào đầu free list** dưới lock. Thao tác O(1).

Việc `memset` được làm **trước khi** lấy lock nên không giữ lock lâu.

2.5 `kalloc()`

```
acquire(&kmem.lock);
r = kmem.freelist;
if(r) kmem.freelist = r->next;     // lấy trang ở đầu danh sách
release(&kmem.lock);
#ifndef LAB_SYSCALL
if(r) memset((char*)r, 5, PGSIZE); // đổ rác 0x05
#endif
return (void*)r;
```

Lấy trang đầu danh sách (O(1)); trả `0` nếu hết bộ nhớ. Caller **luôn phải kiểm tra** kết quả `0` (xem `uvmalloc`, `walk`, `uvmcopy`). Bản debug ghi `0x05` để bắt code giả định trang mới toàn số 0 — vì vậy nơi nào cần trang sạch (page table) đều tự `memset(…, 0, PGSIZE)`.

2.6 Lưu ý về build hiện tại: `LAB_SYSCALL`

conf/lab.mk đặt `LAB=syscall`, và Makefile biến nó thành `-DLAB_SYSCALL`. Do đó, **trong build hiện tại hai đoạn `#ifndef LAB_SYSCALL` bị loại bỏ**: `kfree` và `kalloc` **không** đổ rác `0x01`/`0x05`, và `uvmalloc` (trong `vm.c`) **không** `memset` trang mới về 0. Hệ quả: trang `kalloc` trả về chứa dữ liệu cũ của lần dùng trước, và trang mà `uvmalloc` cấp cho user (heap qua `sbrk`, stack, …) **không được xoá** — một rò rỉ thông tin giữa các process. Đây là hành vi của skeleton MIT lab chứ không phải xv6 gốc; nếu bài của bạn không cần thì có thể bỏ define này. (Theo CLAUDE.md thì cơ chế `LAB=` là phần thừa không thuộc bài của bạn; điểm này chỉ nêu vì nó thật sự đổi hành vi của ba file đang đọc.)

2.7 Hạn chế của thiết kế

- Chỉ cấp phát được **từng trang một**; không có cấp phát vùng liên tục lớn.
- Một free list toàn cục + một lock → các CPU tranh chấp lock (lab `lock` của MIT sửa bằng free list riêng cho mỗi CPU).
- Không có reference count → mỗi trang vật lý chỉ thuộc một chủ (lab `cow` của MIT thêm).

---

3. `kernel/vm.c` — bộ nhớ ảo

3.1 Biến toàn cục

```
pagetable_t kernel_pagetable;   // page table duy nhất của kernel
extern char etext[];            // cuối code kernel (từ kernel.ld)
```

```
extern char trampoline[];       // địa chỉ vật lý của trang trampoline
(trampoline.S)
```

`pagetable_t` là `uint64 *` — con trỏ tới trang chứa 512 PTE.

3.2 Khởi tạo page table của kernel

`kvmmake()`

Tạo page table kernel kiểu **direct map**: địa chỉ ảo = địa chỉ vật lý (trừ trampoline và kernel stack). Cấp một trang cho page table gốc (`kalloc` + `memset 0`) rồi `kvmmap` từng vùng:

Vùng	VA = PA	Quyền	Ghi chú
`UART0` (1 trang)	`0x10000000`	R+W	thiết bị
`VIRTIO0` (1 trang)	`0x10001000`	R+W	thiết bị
`PLIC` (`0x4000000` = 64 MiB)	`0x0C000000`	R+W	thiết bị
Kernel text: `[KERNBASE, etext)`	`0x80000000`	**R+X**	code: đọc/chạy, không ghi → chống ghi đè code
Kernel data + RAM còn lại: `[etext, PHYSTOP)`	`etext`	**R+W**	data, bss, và mọi trang kalloc sẽ cấp (không X)
Trampoline (1 trang)	VA `TRAMPOLINE` → PA `trampoline`	R+X	**không** phải identity map
Kernel stacks	`KSTACK(p)`	R+W	qua `proc_mapstacks()`

(Hai khối `#ifdef LAB_NET` map PCI-E/e1000 chỉ có hiệu lực với `LAB=net`, không biên dịch trong build hiện tại.)

Direct map RAM giúp kernel dễ dùng: từ một địa chỉ vật lý `pa` lấy được con trỏ dùng ngay `(void*)pa`. Toàn bộ `kalloc`, `walk`, `copyin/out` dựa vào điều này.

Không có W+X cùng lúc ở bất kỳ vùng nào — nguyên tắc W^X.

`kvminit()`

Gọi `kvmmake()` và lưu kết quả vào `kernel_pagetable`. Chạy một lần, trên hart 0.

`kvminithart()`

```
sfence_vma();
w_satp(MAKE_SATP(kernel_pagetable));
sfence_vma();
```

Bật phân trang **cho hart hiện tại**. Mỗi hart phải tự gọi (nên được gọi cả ở `main()` của hart 0 và các hart còn lại — kernel/main.c:21, :39). `sfence.vma` đầu: đảm bảo mọi ghi vào page table đã hoàn tất trước khi `satp` đọc; `sfence.vma` sau: xoá TLB cũ.

3.3 Duyệt page table

`walk(pagetable, va, alloc)` — hàm cốt lõi

Trả về **con trỏ tới PTE cấp 0 (lá)** ứng với `va`.

```
if(va >= MAXVA) panic("walk");
for(int level = 2; level > 0; level--) {
  pte_t *pte = &pagetable[PX(level, va)];
  if(*pte & PTE_V) {
    pagetable = (pagetable_t)PTE2PA(*pte);   // đi xuống cấp dưới
  } else {
    if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
      return 0;
    memset(pagetable, 0, PGSIZE);
    *pte = PA2PTE(pagetable) | PTE_V;        // nối page table mới vào
  }
}
return &pagetable[PX(0, va)];
```

Đi từ cấp 2 xuống cấp 1. Ở mỗi cấp: lấy PTE bằng `PX(level, va)`; nếu hợp lệ thì chuyển sang page table con (`PTE2PA`); nếu không hợp lệ thì tuỳ `alloc`:

- `alloc == 0` → trả `0` (không tồn tại).
- `alloc != 0` → `kalloc` một page table mới, xoá về 0, ghi PTE trỏ tới nó (chỉ có `PTE_V`, **không** R/W/X → đúng quy ước "PTE trỏ bảng con"). Cuối cùng trả `&pagetable[PX(0, va)]` ở cấp 0. **PTE đó có thể chưa hợp lệ** — caller phải tự kiểm tra `PTE_V`.

Cách này dùng được vì kernel direct-map RAM: `PTE2PA(*pte)` là địa chỉ vật lý của page table con, và cũng dùng thẳng làm con trỏ.

Hàm trả `0` trong hai trường hợp: không có bảng trung gian và `alloc==0`, hoặc hết bộ nhớ khi `alloc!=0`. Khối `#ifdef LAB_PGTBL` (xử lý superpage) không biên dịch ở build này.

`walkaddr(pagetable, va)`

Dịch `va` → địa chỉ vật lý, trả `0` nếu không map. Gọi `walk(..., 0)` rồi kiểm tra:

1. `va < MAXVA`;
2. PTE tồn tại;
3. `PTE_V` đặt;
4. **`PTE_U` đặt** — hàm chỉ dùng cho **trang user**. Nhờ đó `copyin`/`copyout` không thể bị lừa đọc/ghi bộ nhớ kernel qua con trỏ user.

Lưu ý: trả về địa chỉ vật lý của **trang** (đã bỏ offset); caller cộng offset trong trang.

3.4 Tạo / xoá ánh xạ

`kvmmap(kpgtbl, va, pa, sz, perm)`

Bọc `mappages` cho page table kernel; `panic` nếu lỗi. Chỉ dùng lúc boot, không flush TLB.

`mappages(pagetable, va, size, pa, perm)`

Tạo PTE cho dải `[va, va+size)` trỏ tới `[pa, pa+size)`.

- Yêu cầu `va` và `size` **căn trang**, `size != 0` (nếu không `panic`).
- Với mỗi trang: `walk(..., alloc=1)` lấy PTE (tạo bảng trung gian nếu cần) → `panic("remap")` nếu PTE đã `PTE_V` → ghi `*pte = PA2PTE(pa) | perm | PTE_V`.
- Trả `0` nếu thành công, `-1` nếu `walk` hết bộ nhớ (có thể để lại ánh xạ dở dang; caller tự dọn).

Dùng `last = va + size - PGSIZE` và vòng `for(;;)` + `if(a == last) break` thay vì `a < va+size` để tránh tràn số khi `va+size` chạm đỉnh không gian 64 bit.

`uvmunmap(pagetable, va, npages, do_free)`

Gỡ `npages` ánh xạ bắt đầu từ `va` (căn trang). Với mỗi trang:

- `walk` phải có kết quả, PTE phải `PTE_V` (nếu không `panic`, in `va` và `pte` để debug).
- `PTE_FLAGS(*pte) == PTE_V` nghĩa là PTE không có R/W/X → không phải lá → `panic("not a leaf")`.
- Nếu `do_free`: `kfree` trang vật lý.
- `*pte = 0`.

Không giải phóng các page-table page trung gian — việc đó của `freewalk`. Hàm cũng không tự flush TLB.

3.5 Page table của user process

`uvmcreate()`

Cấp một page table gốc rỗng (`kalloc` + `memset 0`); trả `0` nếu hết bộ nhớ.

`uvmfirst(pagetable, src, sz)`

Nạp `initcode` (chương trình user đầu tiên) vào địa chỉ ảo 0. `sz < PGSIZE`. Cấp một trang, xoá, map tại VA 0 với `PTE_W|PTE_R|PTE_X|PTE_U` rồi `memmove` code vào. Chỉ dùng trong `userinit()`.

`uvmalloc(pagetable, oldsz, newsz, xperm)`

Mở rộng bộ nhớ process từ `oldsz` lên `newsz` (không nhất thiết căn trang). Trả `newsz` hoặc `0` nếu lỗi.

- `newsz < oldsz` → không làm gì, trả `oldsz`.
- `oldsz = PGROUNDUP(oldsz)`, rồi mỗi trang: `kalloc` → (xoá 0 nếu không `LAB_SYSCALL`) → `mappages` với `PTE_R | PTE_U | xperm`.
- `xperm` là quyền thêm do caller chọn: `exec` truyền `PTE_X` hoặc `PTE_W` theo cờ segment ELF; `growproc` truyền `PTE_W` cho heap.

- Lỗi giữa chừng → `uvmdealloc` thu hồi những trang đã cấp, trả `0`. Nếu chỉ `mappages` lỗi thì `kfree(mem)` trang vừa cấp trước.

`uvmdealloc(pagetable, oldsz, newsz)`

Co từ `oldsz` về `newsz`. Chỉ khi `PGROUNDUP(newsz) < PGROUNDUP(oldsz)` mới có trang để bỏ → gọi `uvmunmap(…, do_free=1)`. Trả `newsz`. (Nếu `newsz >= oldsz` trả `oldsz`.)

`freewalk(pagetable)`

Giải phóng đệ quy **các page-table page**. Với mỗi trong 512 PTE:

- Hợp lệ và không có R/W/X → trỏ bảng con → đệ quy rồi `pagetable[i] = 0`.
- Hợp lệ và có R/W/X → còn sót ánh xạ lá → `panic("freewalk: leaf")`. Nghĩa là **phải `uvmunmap` hết lá trước**. Cuối cùng `kfree` chính trang bảng này.

`uvmfree(pagetable, sz)`

Dọn trọn một không gian user: `uvmunmap(0, PGROUNDUP(sz)/PGSIZE, free=1)` (giải phóng trang dữ liệu), rồi `freewalk` (giải phóng bảng). Gọi bởi `proc_freepagetable` khi process kết thúc hoặc khi `exec` thất bại.

`uvmcopy(old, new, sz)`

Dùng trong `fork()`: sao chép không gian bộ nhớ cha sang con. Với mỗi trang trong `[0, sz)`:

1. `walk(old)` phải tồn tại và `PTE_V` (nếu không `panic`).
2. Lấy `pa`, `flags` (giữ nguyên quyền).
3. `kalloc` trang mới, `memmove` **toàn bộ 4096 byte** từ trang cha.
4. `mappages(new, i, …, flags)`.
5. Lỗi → `uvmunmap(new, 0, i/PGSIZE, 1)` dọn phần đã làm, trả `-1`.

Đây là **sao chép đầy đủ (eager copy)**, không copy-on-write: `fork` tốn thời gian và RAM tỷ lệ với kích thước process. Trong file có dòng `szinc = PGSIZE;` bị lặp hai lần (dòng 347–348): vô hại, thừa.

`uvmclear(pagetable, va)`

Xoá bit `PTE_U` của trang tại `va`. `exec` dùng để biến trang ngay dưới user stack thành **guard page**: user không truy cập được, nên stack tràn gây page fault.

3.6 Sao chép dữ liệu kernel ↔ user

Kernel không được dùng trực tiếp con trỏ do user cung cấp (page table đang dùng ở kernel mode không chứa user mapping, và user có thể đưa địa chỉ bậy). Ba hàm sau dịch bằng page table của process rồi copy qua địa chỉ vật lý (dùng được vì kernel direct-map RAM).

`copyout(pagetable, dstva, src, len)` — kernel → user

Lặp theo trang:

1. `va0 = PGROUNDDOWN(dstva)`; `va0 >= MAXVA` → `-1`.

2. `walk(..., 0)` lấy PTE; không có → `-1`.
3. **Bản này có kiểm tra `(*pte & PTE_W) == 0 → return -1`** — cấm ghi lên trang user chỉ-đọc (ví dụ vùng text). (Đây là kiểm tra có trong xv6-riscv mới; xv6 cũ không có.)
4. `walkaddr` → `pa0` (đồng thời kiểm `PTE_V` và `PTE_U`).
5. `n` = số byte còn trong trang, cắt theo `len`; `memmove` vào `pa0 + offset`.
6. Cập nhật `len`, `src`, `dstva = va0 + PGSIZE`.

Dải copy có thể vắt qua nhiều trang, mỗi trang có thể ở khung vật lý không liền kề — vòng lặp xử lý điều này.

`copyin(pagetable, dst, srcva, len)` — user → kernel

Giống `copyout` nhưng chiều ngược lại và không cần kiểm quyền ghi: `walkaddr` (bắt buộc `PTE_U`) rồi `memmove` từ `pa0 + offset` vào `dst`. Trả `-1` nếu trang không map.

`copyinstr(pagetable, dst, srcva, max)` — chuỗi C từ user

Chép tới khi gặp `'\0'` hoặc hết `max` byte. Bên trong mỗi trang, duyệt từng byte: gặp `'\0'` thì ghi `'\0'` vào `dst`, đặt `got_null` và dừng. Trả `0` nếu thấy `'\0'`, `-1` nếu trang không map hoặc chuỗi dài quá `max` mà không có kết thúc. Dùng cho `argstr` (đường dẫn file, đối số `exec`, ...).

3.7 Code chỉ có trong skeleton lab

`vmprint` và `pgpte` (cuối file) nằm trong `#ifdef LAB_PGTBL` → không biên dịch ở build hiện tại (và `vmprint` còn là hàm rỗng "your code here").

---

4. Ghép lại: luồng cấp phát bộ nhớ

Boot (`main()` hart 0):

1. `kinit()` — đưa RAM `[end, PHYSTOP)` vào free list.
2. `kvminit()` → `kvmmake()` — dựng page table kernel (direct map + trampoline + kernel stack), mỗi bảng lấy từ `kalloc`.
3. `kvminithart()` — ghi `satp`, bật phân trang. Từ đây mọi địa chỉ là địa chỉ ảo.

Cấp thêm heap cho process (`sbrk` → `growproc` → `uvmalloc`):

```
uvmalloc
 ├─ kalloc()                lấy 1 trang vật lý từ free list
 └─ mappages(...)
     └─ walk(alloc=1)       tạo bảng trung gian nếu thiếu (kalloc)
        ghi PTE = PA2PTE(pa) | R | U | W | V
```

Giải phóng khi process thoát (`uvmfree`):

```
uvmunmap(do_free=1)  → kfree từng trang dữ liệu, xoá PTE lá
freewalk             → kfree từng page-table page (đệ quy)
```

Syscall nhận con trỏ từ user (ví dụ `read(fd, buf, n)`):

```
sys_read → copyout(p->pagetable, buf, kbuf, n)
              └─ walk + walkaddr → pa → memmove
```

5. Bảng tra nhanh hàm

Hàm	File	Làm gì	Trả về
`kinit`	kalloc.c	Khởi tạo allocator, nạp RAM vào free list	—
`freerange`	kalloc.c	`kfree` mọi trang trong một dải	—
`kfree`	kalloc.c	Trả một trang về free list	—
`kalloc`	kalloc.c	Lấy một trang từ free list	con trỏ hoặc 0
`kvmmake` / `kvminit`	vm.c	Dựng page table kernel	`pagetable_t` / —
`kvminithart`	vm.c	Bật paging trên hart hiện tại	—
`walk`	vm.c	Tìm (và tuỳ chọn tạo) PTE lá của `va`	`pte_t*` hoặc 0
`walkaddr`	vm.c	`va` user → `pa`, kiểm `V` và `U`	`pa` hoặc 0
`kvmmap`	vm.c	`mappages` cho kernel, panic nếu lỗi	—
`mappages`	vm.c	Tạo PTE cho một dải	0 / -1
`uvmunmap`	vm.c	Gỡ ánh xạ, tuỳ chọn free trang	—
`uvmcreate`	vm.c	Page table user rỗng	`pagetable_t` hoặc 0
`uvmfirst`	vm.c	Nạp initcode vào VA 0	—
`uvmalloc` / `uvmdealloc`	vm.c	Tăng / giảm bộ nhớ user	size mới (`uvmalloc` trả 0 nếu lỗi)
`freewalk`	vm.c	Giải phóng page-table page (đệ quy)	—
`uvmfree`	vm.c	Giải phóng toàn bộ bộ nhớ + bảng của user	—
`uvmcopy`	vm.c	Copy bộ nhớ cha → con (fork)	0 / -1
`uvmclear`	vm.c	Bỏ `PTE_U` (guard page)	—
`copyout`	vm.c	kernel → user	0 / -1
`copyin`	vm.c	user → kernel	0 / -1
`copyinstr`	vm.c	chuỗi user → kernel	0 / -1